LFSR taps, shifts and old-value timing

Connect the 5-bit, schematic-based 3-bit and 32-bit LFSRs through one method: derive next-bit equations from old state, translate one-based taps, then verify the cycle.

Kapil Tripathi | Verilog & digital design | 12 pages

CONTENTS / click a row to jump

KEEP THIS IDEA

Treat every LFSR as simultaneous next-state equations. Convert tap position k to q[k-1], keep one clocked owner, and never seed an XOR-feedback LFSR with zero.

Tracker entries: 141, 145, 151

Study notes by Kapil Tripathi, based on HDLBits exercises. Original questions, source references and dated verification records are retained in the notes.

ALL NOTES + ENTRY INDEX HDLBits problem collection

SERIES MAP / ENTRIES 141, 145, 151

Three LFSRs, one translation method

These exercises move from a drawn five-bit Galois LFSR, through a three-bit mux-and-flip-flop circuit, to a 32-bit Galois LFSR. The reliable workflow is always the same: name the old state, write each next-bit equation, and only then compress the equations into Verilog.

Entry	Problem	Main reasoning task
141	Lfsr5	Translate one-based taps 5 and 3 into a right-shifting Galois update.
145	Mt2015_lfsr	Read the mux diagram, distinguish parallel load from feedback, and map board pins.
151	Lfsr32	Scale the same Galois pattern to taps 32, 22, 2, and 1 without an index error.

Questions this note answers

What does an LFSR actually store, and why can a small register generate a long sequence without using a counter?

If the diagram says tap position 3, why does the code override q[2]? Why does tap 22 become q[21] in the 32-bit version?

Why is a whole-vector nonblocking assignment followed by a few slice assignments valid? Do the XOR expressions see old q or the partly shifted q?

For Mt2015_lfsr: what exactly is the question asking, how do L, R, Clock, Q, SW, KEY, and LEDR correspond, and why does the accepted solution work?

All three open HDLBits result panels showed Status: Success! on 9 September 2026. The note also verifies the full 31-state and 7-state cycles locally.

SHARED MODEL / STATE RECURRENCE

An LFSR is a state machine written as bit equations

An n-bit register stores one state. At each active clock edge, every next bit is a linear XOR expression of the old bits. Repeating that fixed transformation produces a deterministic orbit through the state space.

The three layers

Layer	Question to ask	Example
State	What values exist before this edge?	old q[4:0] = 00001
Next-state equations	What should each destination bit become?	next q[2] = old q[3] ^ old q[0]
Clocked storage	When do all those values become visible?	Together, immediately after posedge clk

Why XOR is called linear

Over one-bit arithmetic, XOR is addition modulo 2: 0^0=0, 0^1=1, 1^0=1, and 1^1=0. Every LFSR next-state bit is a sum of selected old bits with no carries. That is why matrix and polynomial methods can analyze its period.

Maximum length and the missing state

An n-bit register has 2^n possible bit patterns. A maximum-length XOR-feedback LFSR visits all 2^n - 1 nonzero states before repeating. It cannot include zero because XORs of all zeros are still zero; 000...0 is a separate self-loop called lockup.

Resetting to 1 is therefore functional, not decorative. It chooses a nonzero seed. Any other nonzero seed on the same maximal cycle changes the starting point but not the cycle length.

LFSR is not the same as binary count

The output sequence looks irregular and is useful for test patterns, scrambling, CRC-style logic, and compact counters. It is still fully deterministic. Knowing the current state determines every future state.

Recall test

If q is all zero and the only feedback operators are XOR, no clock edge can escape zero.

SHARED MODEL / TAP NUMBERING

Tap position k maps to Verilog index k-1

HDLBits numbers tap positions from 1, but Verilog vectors use zero-based indices. Write the position and index on separate lines before coding; this single step prevents the most common LFSR error.

Named position	Verilog bit	Five-bit meaning	Thirty-two-bit meaning
1	q[0]	Output/feedback bit	Output plus tap stage
2	q[1]	Ordinary shift stage	Tap stage
3	q[2]	Tap stage	Ordinary shift stage
22	q[21]	Not present	Tap stage
32	q[31]	Not present	Feedback enters here

Start from the ordinary right shift

```
next_q = {old_q[0], old_q[N-1:1]};
// next_q[N-1] = old_q[0]
// next_q[i]   = old_q[i+1] for 0 <= i < N-1
```

Then override the tap destinations

For a Galois tap at position k below the top position, the destination q[k-1] receives its ordinary shifted source XORed with the outgoing feedback bit q[0]. Thus position 3 gives next q[2] = old q[3] ^ old q[0]. Position 22 gives next q[21] = old q[22] ^ old q[0].

Why the top tap may not need a separate line

The base concatenation already writes old q[0] into the top stage. The diagram often draws the top XOR with a constant-zero second input for consistency. XOR with zero changes nothing, so the whole-vector shift already implements it.

Mechanical translation

Diagram position -> subtract 1 -> destination index. Ordinary shifted source -> add 1 -> old source index. XOR the outgoing old q[0] only at tapped destinations.

ENTRY 141 / LFSR5

The five-bit Galois solution

The circuit shifts right. Position 5 receives the outgoing q[0], while the tap at position 3 receives old q[3] XOR old q[0]. The synchronous reset selects seed 00001.

```
module top_module (
    input clk,
    input reset,              // Active-high synchronous reset
    output reg [4:0] q
);
    always @(posedge clk) begin
        if (reset) begin
            q <= 5'h01;
        end else begin
            q    <= {q[0], q[4:1]};
            q[2] <= q[3] ^ q[0];
        end
    end
endmodule
```

Line-by-line meaning

Code	Next-state equation
q <= {q[0], q[4:1]};	q4'=q0, q3'=q4, q2'=q3, q1'=q2, q0'=q1
q[2] <= q[3] ^ q[0];	Replace the base q2'=q3 with tapped q2'=q3^q0
q <= 5'h01;	Select nonzero seed 00001 when reset is high at the edge

Why q[2] is assigned twice

Both assignments are in the same always block. The whole-vector line supplies the default next value for every bit, then the later slice assignment replaces only q[2]. This is a compact default-plus-exception pattern, not two hardware drivers.

Why output reg is shown

The always block stores q, so q must be a procedural variable in Verilog. Declaring output reg [4:0] q makes the port itself the register. An internal reg plus assign q = internal_q would also be valid but adds an unnecessary name here.

Source: HDLBits Lfsr5. The page specifies taps 5 and 3, reset to 1, and shows Status: Success!

ENTRY 141 / LFSR5 TRACE

Old values produce the 31-state cycle

Tracing from reset seed 01 hex exposes both the shift direction and the tap override. It also matches the beginning of HDLBits' displayed reference waveform.

Step	old q	old q0	base shift	new q2	next q
0	00001 (01)	1	10000	q3^q0 = 1	10100 (14)
1	10100 (14)	0	01010	q3^q0 = 0	01010 (0A)
2	01010 (0A)	0	00101	q3^q0 = 1	00101 (05)
3	00101 (05)	1	10010	q3^q0 = 1	10110 (16)
4	10110 (16)	0	01011	q3^q0 = 0	01011 (0B)

The important first edge

Starting at 00001, an ordinary rotate-like right shift would produce 10000. The position-3 tap changes q[2] from old q[3]=0 to 0^1=1, so the real result is 10100. If your first post-reset state is not 10100, the shift direction or tap index is wrong.

A compact sequence fingerprint

```
01 -> 14 -> 0A -> 05 -> 16 -> 0B -> 11 -> 1C -> 0E -> ...
```

Local exhaustive check

The builder starts at 00001, records states until the first repeat, and asserts 31 unique states, no zero state, and a return to 00001. That verifies the implemented recurrence, not merely the first few waveform labels.

Reset timing

Because reset is inside always @(posedge clk), it is synchronous. A high reset between edges does not immediately change q. At the next rising edge, reset wins over the feedback update and q becomes 00001.

Debug boundary

Check one edge after reset first. A correct 00001 -> 10100 transition proves the seed, shift direction, feedback bit, and tap destination together.

ENTRY 145 / MT2015_LFSR

Read the schematic as three D-input equations

Each flip-flop stores one LEDR bit. Its input mux chooses either the parallel R input or the feedback-network value. The load selector L is shared, so all three bits load or advance together.

Schematic label	Top-level port	Role
R[2:0]	SW[2:0]	Three-bit value loaded in parallel
L	KEY[1]	Mux selection: 1 loads R, 0 selects feedback
Clock	KEY[0]	Rising edge stores all three selected D inputs
Q[2:0]	LEDR[2:0]	Visible stored state

Feedback equations when L=0

```
next Q[0] = old Q[2]
next Q[1] = old Q[0]
next Q[2] = old Q[1] ^ old Q[2]
```

Parallel-load equations when L=1

```
next Q[0] = R[0]
next Q[1] = R[1]
next Q[2] = R[2]
```

What the question is asking

The task is not asking you to instantiate physical switches, keys, LEDs, muxes, or flip-flop submodules. It asks for behavior equivalent to the shown circuit while using the required board-facing port names. One clocked always block implements all three flip-flops and their shared load selection.

Do not infer an extra reset

This module has no reset port. KEY[1] and SW supply an explicit load path instead. To establish a known state, set SW to a chosen seed, use L=1 for one active clock edge, then use L=0 to advance the recurrence.

Source: HDLBits Mt2015_lfsr. The page maps R to SW, Clock to KEY[0], L to KEY[1], and Q to LEDR.

ENTRY 145 / MT2015_LFSR RTL

Load has explicit priority over feedback

The if branch models the shared mux select. At every rising edge of KEY[0], KEY[1] chooses one complete three-bit next state: SW for load, or the feedback equations for advance.

```
module top_module (
    input  [2:0] SW,          // Parallel-load value R
    input  [1:0] KEY,         // KEY[1] = L, KEY[0] = clock
    output reg [2:0] LEDR     // Register outputs Q
);
    always @(posedge KEY[0]) begin
        if (KEY[1]) begin
            LEDR <= SW;
        end else begin
            LEDR[0] <= LEDR[2];
            LEDR[1] <= LEDR[0];
            LEDR[2] <= LEDR[1] ^ LEDR[2];
        end
    end
endmodule
```

Why LEDR <= SW is the right load

The vector assignment is exactly the three parallel equations LEDR[i] <= SW[i]. No shifting happens in that branch. Since it is the if branch, it has priority whenever KEY[1] is 1 at the sampling edge.

Why all three feedback assignments read old LEDR

Nonblocking <= evaluates all three right-hand sides from the same old LEDR. The first line does not alter the value read by the second, and neither alters the operands read by the XOR. They represent three flip-flops sampling simultaneously.

At posedge KEY[0]	KEY[1]	Stored result
Load operation	1	LEDR_next = SW
Sequence operation	0	LEDR_next = {LEDR1^LEDR2, LEDR0, LEDR2}

Board polarity versus logic polarity

The exercise connects KEY[1] directly to logical L and the accepted circuit loads when that signal is 1. A physical pushbutton may be electrically active-low on the board; that affects how a person drives the pin, not the Boolean behavior requested by the schematic. Do not silently insert an inversion that the problem does not show.

Original saved question

The editor comment asked to understand the question before the solution. The mapping table on page 7 and the two equation sets on page 7 are the essential pre-code interpretation; this page is the direct RTL transcription.

ENTRY 145 / MT2015_LFSR TRACE

Seven nonzero states prove the recurrence

Load seed 001, deassert L, and advance once per rising edge. The circuit visits all seven nonzero three-bit patterns before returning to 001.

Old Q	Q0'=old Q2	Q1'=old Q0	Q2'=old Q1^old Q2	Next Q
001	0	1	0^0=0	010
010	0	0	1^0=1	100
100	1	0	0^1=1	101
101	1	1	0^1=1	111
111	1	1	1^1=0	011
011	0	1	1^0=1	110
110	1	0	1^1=0	001

Why 000 never appears

Every listed nonzero state maps to another nonzero state. Separately, 000 maps to 000 because each next bit is either an old bit or XOR of old bits. Loading 000 therefore locks the generator. Load any of the seven nonzero seeds to enter the same seven-state cycle at a different point.

What would blocking assignments risk?

If the feedback branch used = and the statements executed sequentially, later equations could read values already changed by earlier statements. The simulated recurrence would depend on source order and no longer match three simultaneous D flip-flops. Nonblocking <= preserves the intended old-state snapshot.

A useful mental rewrite

```
old = LEDR;
LEDR_next[0] = old[2];
LEDR_next[1] = old[0];
LEDR_next[2] = old[1] ^ old[2];
```

You do not need this temporary variable in the final RTL. It is a reasoning device that makes simultaneous sampling explicit.

Verification result

001 -> 010 -> 100 -> 101 -> 111 -> 011 -> 110 -> 001. Period = 7 = 2^3 - 1.

ENTRY 151 / LFSR32

The 32-bit solution is the five-bit pattern scaled up

Start with the same right-shift default, then override destinations for the lower taps. Positions 22, 2, and 1 become q[21], q[1], and q[0]. Position 32 is already handled by feedback into q[31].

```
module top_module (
    input clk,
    input reset,              // Active-high synchronous reset
    output reg [31:0] q
);
    always @(posedge clk) begin
        if (reset) begin
            q <= 32'h00000001;
        end else begin
            q     <= {q[0], q[31:1]};
            q[21] <= q[22] ^ q[0];
            q[1]  <= q[2]  ^ q[0];
            q[0]  <= q[1]  ^ q[0];
        end
    end
endmodule
```

Tap position	Destination index	Ordinary source	Implemented next value
32	q[31]	constant 0 at drawn XOR	old q[0]
22	q[21]	old q[22]	old q[22] ^ old q[0]
2	q[1]	old q[2]	old q[2] ^ old q[0]
1	q[0]	old q[1]	old q[1] ^ old q[0]

The off-by-one trap

Writing q[22], q[2], and q[1] treats the one-based tap labels as if they were Verilog indices. That moves every lower XOR one stage too high. The first visible state can expose the error immediately.

First edge after reset seed 00000001

```
base shift: q[31]=1, all other bits=0
tap overrides: q[21]=1, q[1]=1, q[0]=1
next q = 32'h80200003
```

Source: HDLBits Lfsr32. The page specifies taps 32, 22, 2, and 1 and reset to 32'h1.

ENTRY 151 / NONBLOCKING OVERRIDES

The XORs read old q, never a partly shifted q

The accepted compact style relies on two Verilog rules: nonblocking right-hand sides are sampled before updates become visible, and the last assignment to an overlapping destination in one process determines that destination's scheduled value.

Statement	RHS snapshot	Scheduled destination
q <= {q[0],q[31:1]}	Entire old q	All 32 next bits
q[21] <= q[22]^q[0]	old q[22], old q[0]	Replace scheduled bit 21
q[1] <= q[2]^q[0]	old q[2], old q[0]	Replace scheduled bit 1
q[0] <= q[1]^q[0]	old q[1], old q[0]	Replace scheduled bit 0

This is one owner, not multiple drivers

All four assignments live in one clocked process. Synthesis sees one bank of 32 flip-flops with combinational next-value logic. A separate always block or continuous assign writing q would create another owner and would be a real multiple-driver error.

Equivalent explicit next-vector style

```
reg [31:0] next_q;
always @(*) begin
    next_q     = {q[0], q[31:1]};
    next_q[21] = q[22] ^ q[0];
    next_q[1]  = q[2]  ^ q[0];
    next_q[0]  = q[1]  ^ q[0];
end
always @(posedge clk)
    if (reset) q <= 32'h1; else q <= next_q;
```

Both styles describe the same hardware. The explicit form can be easier to audit because every exception names next_q; the compact form is shorter and safe when all overlapping nonblocking assignments remain in one process.

Polynomial check

The stated taps correspond to x^32 + x^22 + x^2 + x + 1, up to the reciprocal orientation used by the right-shifting implementation. The builder verifies the primitive-order conditions using the prime factors 3, 5, 17, 257, and 65537 of 2^32 - 1. This supports the maximal nonzero period stated by the exercise.

REFERENCE / VERIFICATION AND DEBUGGING

Use equations first, code second

The shortest reliable LFSR solution starts from a paper trace. Treat every HDLBits drawing as a next-state specification, then use one clocked owner to store the result.

Check	What to write or test
1. Direction	For each destination, identify the old source before thinking about taps.
2. Numbering	Convert one-based position k to zero-based q[k-1].
3. Feedback	Mark the outgoing old bit once; use it only at drawn tap stages.
4. Storage	Use <= in one edge-triggered owner. Omitted assignment means hold.
5. Seed	Avoid zero for XOR-feedback operation; match the stated reset or load.
6. First edge	Predict one post-seed value by hand and compare it exactly.
7. Period	For small designs, enumerate until repeat and assert zero is absent.

Three reference results

Problem	Seed or load	First result / verified cycle
Lfsr5	00001	10100; all 31 nonzero states, then 00001
Mt2015_lfsr	001 via SW	010; seven-state cycle ending 110 -> 001
Lfsr32	00000001	80200003; primitive-polynomial order check passes

Mistake-to-symptom map

Symptom	Likely cause
First Lfsr5 result is 10000	The tap-3 override was omitted.
Lfsr32 lower XORs appear one stage high	Tap positions were used directly as indices.
Mt2015 result changes with statement order	Blocking = was used for clocked state updates.
Sequence stays zero	Zero was reset or loaded into an XOR-feedback LFSR.
Compiler rejects q as an l-value	The procedurally assigned output was not declared reg/variable.

Verification used for this note

The builder exhaustively verifies the 5-bit period of 31 and the 3-bit period of 7, checks their exact early sequences, checks the 32-bit first transition 00000001 -> 80200003, and proves the stated 32-bit polynomial has full multiplicative order. The three current HDLBits panels independently show accepted submissions.